Disciplina: Técnicas Avançadas de Banco de Dados Relacional e Não Relacional

Store Procedures – Estrutura de Repetição

Profª. Drª. Giovana Angélica Ros
giovana.ros@cps.sp.gov.br

Fatec
Presidente Prudente

Centro Paula Souza

Estrutura de repetição

- Estruturas de repetição (loops) são fundamentais quando é necessário processar registros um a um, ou executar uma lógica várias vezes até que uma condição seja atingida.
- Diferente de linguagens como Python, C#, ou PHP, no MySQL as repetições ocorrem obrigatoriamente dentro de um bloco BEGIN...END
- Tem 3 laços de repetição:
  - WHILE
  - REPEAT
  - LOOP

WHILE - Sintaxe

- WHILE - testa a condição antes de executar o bloco de código
- Sitaxe:

```
WHILE condição DO
    ...
END WHILE;
```

- Exemplo:

```
DECLARE v_contador INT DEFAULT 1;
WHILE v_contador <= 5 DO
    -- alguma lógica
    SET v_contador = v_contador + 1; -- Incremento
END WHILE;
```

WHILE

- Exemplo: Inserir vários registros de teste de uma vez

```
DELIMITER $$
DROP PROCEDURE IF EXISTS pr_gerar_registros_teste $$
CREATE PROCEDURE pr_gerar_registros_teste(IN p_quantidade INT)
BEGIN
   DECLARE v_contador INT DEFAULT 1;
   WHILE v_contador <= p_quantidade DO
      INSERT INTO produtos (nome, preco, estoque)
      VALUES (CONCAT('Produto Teste ', v_contador), 10.00, 1);
      SET v_contador = v_contador + 1; -- incremento
   END WHILE;
END $$
DELIMITER ;
CALL pr_gerar_registros_teste(2);
```

id	nome	preco	estoque
1	Notebook Pro	4500.00	15
2	Mouse Wireless	120.00	50
3	Monitor 24pol	900.00	8
4	Produto Teste 1	10.00	1
5	Produto Teste 2	10.00	1

REPEAT - Sintaxe

- Executa o bloco de código ao menos uma vez e testa a condição **ao final**. Ele continua repetindo **até que** a condição seja verdadeira (UNTIL)
- Sintaxe:

```
REPEAT
   -- Alguma lógica
   UNTIL condição
END REPEAT;
```

REPEAT

- Exemplo: Inserir vários registros de teste de uma vez

```
DELIMITER $$
DROP PROCEDURE IF EXISTS pr_gerar_produtos_repeat $$
CREATE PROCEDURE pr_gerar_produtos_repeat(IN p_limite INT)
BEGIN
    DECLARE v_contador INT DEFAULT 1;
    REPEAT
        -- O código dentro do REPEAT sempre executa ao menos 1 vez
        INSERT INTO produtos (nome, preco, estoque)
        VALUES (CONCAT('Produto Repeat ', v_contador), 50.00, 10);
        SET v_contador = v_contador + 1;
    -- ATENÇÃO: A condição aqui é de PARADA (Até que...)
    -- No WHILE era "Enquanto for menor", aqui é "Até que seja maior"
    UNTIL v_contador > p_limite
    END REPEAT;
END $$
DELIMITER ;
CALL pr_gerar_produtos_repeat(2);
```

LOOP - Sintaxe

- É um loop "infinito" por padrão. Deve ser usado o comando LEAVE (equivalente ao break) para sair dele baseado em um rótulo (label)
- Sintaxe:

```
meu_loop: LOOP
   --Alguma lógica
   IF condição  THEN
      LEAVE meu_loop; -- Sai do loop
   END IF;
END LOOP meu_loop;
```

LOOP

- Exemplo: Inserir vários registros de teste de uma vez

```
DELIMITER $$
DROP PROCEDURE IF EXISTS pr_gerar_com_loop_puro $$
CREATE PROCEDURE pr_gerar_com_loop_puro(IN p_maximo INT)
BEGIN
   DECLARE v_contador INT DEFAULT 1;
   -- Nomeie o loop (ex: meu_bloco)
   meu_bloco: LOOP
      -- 1. A Condição de Saída (Equivalente ao que o WHILE faz no topo)
      IF v_contador > p_maximo THEN
         LEAVE meu_bloco; -- Aqui o código "sai" para fora do loop
      END IF;
      -- 2. A Ação
      INSERT INTO produtos (nome, preco, estoque)
      VALUES (CONCAT('Item Loop ', v_contador), 5.99, 100);
      -- 3. O Incremento
      SET v_contador = v_contador + 1;
   END LOOP meu_bloco;
END $$
DELIMITER ;
CALL pr_gerar_com_loop_puro(2);
```

Atenção

- Cuidado com o Loop Infinito: No MySQL, um loop infinito pode travar o processamento do servidor. Sempre garanta que a variável de controle (ex: v_contador) está sendo atualizada dentro do bloco.

- Performance: Loops são úteis, mas se você puder resolver com um único UPDATE ou INSERT INTO ... SELECT, use o comando direto. O SQL é otimizado para trabalhar com conjuntos de dados, não linha por linha.

Exercícios

1. Use WHILE. Automação de Dados. Crie uma procedure que recebe um número N, como parâmetro e insira esses N registros na tabela logs_acesso (id, mensagem, data_registro). No insert, use a função CONCAT() e a NOW(). CALL pr_popularLogs(10);

id	mensagem	data_registro
1	Log de teste número 1	2026-03-19 09:50:37
2	Log de teste número 2	2026-03-19 09:50:37
3	Log de teste número 3	2026-03-19 09:50:37
4	Log de teste número 4	2026-03-19 09:50:37
5	Log de teste número 5	2026-03-19 09:50:37
6	Log de teste número 6	2026-03-19 09:50:37
7	Log de teste número 7	2026-03-19 09:50:37
8	Log de teste número 8	2026-03-19 09:50:37
9	Log de teste número 9	2026-03-19 09:50:37
10	Log de teste número 10	2026-03-19 09:50:37

2. Use REPEAT. Sistema de Cobrança. Crie uma procedure que gera parcelas de um pagamento para um cliente de forma automática, na tabela parcelas (id, valor_parcela, data_vencimento). No insert, use a função **DATE_ADD()**, que serve para somar intervalos de tempo a uma data específica, a função **CURDATE()** que retorna a **data atual** do sistema (hoje) e o **INTERVAL v_contador MONTH**, onde ocorrerá o parcelamento, por exemplo, no **1º passo** do loop, v_contador é 1, então ele soma **1 mês** à data de hoje, no **2º passo**, v_contador é 2, então ele soma **2 meses** à data de hoje. VALUES (v_valor_dividido, DATE_ADD(CURDATE(), INTERVAL v_contador MONTH)

CALL pr_gerarParcelas(5, 1000.00);

id	valor_parcela	data_vencimento
1	200.00	2026-04-19
2	200.00	2026-05-19
3	200.00	2026-06-19
4	200.00	2026-07-19
5	200.00	2026-08-19

Exercícios

3. Use LOOP e LEAVE. Processamento de Bônus de Funcionários. Crie uma procedure para percorrer uma lista de IDs e atualizar o salário de funcionários, parando o processo assim que atingirmos um limite de bônus distribuído. No UPDATE, na tabela funcionários (id, nome, salário) some o salário mais o bônus.

CALL pr_distribuirBonus(500.00, 3);

id	nome	salario
1	Ana	2000.00
2	Bruno	2500.00
3	Carla	3000.00
4	Diego	3500.00

id	nome	salario
1	Ana	2500.00
2	Bruno	3000.00
3	Carla	3500.00
4	Diego	3500.00